

Notes on *how* *I built it.*

A landing page for Novi, a project and task management tool for small, fast moving teams. These are the questions I'd expect someone reviewing the work to ask, and my answers, including the parts I'm less sure about and the things I left out.

Starting points

What I was working from.

Q. What did the brief ask for?

A hero with navigation and two calls to action, a feature section, and a footer with navigation, signup and social links. Responsive across phones, tablets and desktops. The design was left up to me, and the brief said to experiment with layouts, modals, animations and interactions beyond the core requirements, so I took that at face value.

Q. Who did you write the page for?

Someone running a team of about five to fifteen people who already pays for three or four tools and is not looking for a fifth. They'll skim, not read.

That drove two things. The headline uses the problem rather than the product category, and the product itself appears in the first screen so there's something to look at before any reading is required. I also kept the free tier visible in several places, since this kind of buyer signs up on their own rather than booking a call.

THE TRADE

There's very little here for a bigger company. No security page, nothing about procurement, one line about SSO in the top plan. If Novi were real that would need to exist somewhere, just not on this page.

Design

The look, and why it isn't dark.

Q. Why light and editorial rather than the usual dark SaaS page?

The brief describes Novi as "one calm workspace". I wanted the page to look like that phrase, so I used a warm off-white background, a serif for the headings, a lot of whitespace and one accent colour.

The other reason is more practical. Almost every project tool ships a near-black page with a glowing gradient. Doing something else makes the submission easier to remember, and I'd rather be judged on a direction I chose than on how well I copied Linear.

THE TRADE

Light backgrounds show every mistake. Uneven spacing, weak contrast and misaligned edges are all obvious, where a dark page would have hidden them. Most of my time went into spacing and hairlines because of it.

Q. Where did the accent colour come from?

Veel's logo. I pulled `#4119F4` out of the PNG attached to the brief. It seemed like a reasonable nod to the company reviewing it without putting their branding on a different product.

In dark mode it becomes `#7C5CFF`. The original is too dark to read against a near-black background.

Q. Why three typefaces?

Instrument Serif for headings, Inter for body and interface text, JetBrains Mono for small labels and keyboard hints. Three families is more than I'd normally use, but each one is doing a different job and the mono in particular makes UI labels look like UI rather than prose.

All three load through `next/font`, so they're self-hosted and the metrics are inlined. Nothing reflows when they arrive.

One thing to know if you touch the type: Instrument Serif only ships a 400 weight. Any `font-bold` on a heading renders as a fake synthetic bold, so emphasis has to come from italics. I put a rule in the stylesheet to keep that from happening by accident.

The board in the hero

The part that took the longest.

Q. Why is it a working board instead of a screenshot?

The brief asked for "a supporting image or graphic". A screenshot would have satisfied that, but you can't tell anything about how a tool feels from a picture of it. Since the assessment mentions interactive components, building the thing seemed like a better answer than drawing it.

It's three columns with cards you can drag, and tabs for Board, Threads and Timeline. I ended up reusing the same component in the walkthrough section and inside the demo dialog, driven by a `view` prop, so the cost was spread across three places.

THE TRADE

This is easily the most expensive thing on the page and it's where both dependencies came from. If I'd been short on time it's the first thing I'd have swapped for an image.

Q. Why dnd-kit?

I looked at three options. Native HTML5 drag and drop doesn't work on touch and you can't style the drag image, which rules it out. Motion has a reorder helper, but it only handles reordering within one list, and moving a card between columns is the whole point.

dnd-kit does both and includes keyboard dragging, which I wouldn't have written myself. You can tab to a card, pick it up with space, move it with the arrow keys and drop it. I did rewrite the screen reader announcements, though. By default they read the internal id, so you'd hear "t-1 is over doing". Now it says the task title and column name.

THE TRADE

About 12 kB gzipped. It also puts `role="button"` on every draggable, which meant the columns couldn't stay as a `<ul>`, because a list item with a button role isn't a valid list item. They're plain divs now.

Q. Why do the tabs behave differently depending on where the board appears?

Because something different owns the view in each case. In the hero the component keeps its own state and a tab click switches the view. In the walkthrough the view comes from scroll position, so clicking a tab scrolls to that step instead. In the demo dialog it moves the step, the same as the dots underneath.

This started as a bug. The walkthrough and the dialog both passed a view down without passing anything to change it, so the tabs rendered, highlighted on hover, and did nothing when clicked. I only caught it by clicking them.

Animation

Every moving thing on the page and why it moves that way.

Q. Is there a system to it, or is each animation its own thing?

There's a system. Four easing curves and three springs live in one file and everything pulls from there. The one I use most is an ease-out-expo curve, which moves quickly at the start and takes a long time to settle. It makes things look like they're arriving rather than being shoved into place.

I use springs only where you're directly manipulating something: the nav indicator, the tab pill, a card you've just dropped. Springs have a sense of weight to them, which suits those and doesn't suit a paragraph fading in.

The point of keeping them in one file is that I stop making the decision individually. Otherwise you end up with fourteen slightly different durations.

Q. Why do the section reveals play again when you scroll back up?

Most implementations fire once and stop. That's fine the first time, but if you scroll back up to re-read something, the page is just sitting there fully arrived, and reloading is the only way to see it again.

So each section watches its own position and fades back out on the way off screen. Coming in takes about 600 ms, going out about 300 ms. I tried matching them and the slow exit looked broken, like the page was struggling.

Elements move about 20 px. I started at 40 and it was too much: your eye follows the movement instead of reading the text.

Q. What's going on with the buttons?

Each one has a 2 px solid edge underneath it in a darker shade. Hovering lifts the button and deepens that edge, clicking collapses it and moves the button down, so it behaves a bit like a physical key.

I did this because a colour change on click is easy to miss, especially on a slow connection where nothing else happens for a second. The press is 75 ms against 200 ms for the hover. A press that takes as long as a hover feels laggy.

The hover scale is 1.5%. I had it at 5% first and the text visibly blurred during the transition.

Q. Why is the walkthrough driven by scroll instead of buttons?

It's the one section explaining a sequence, and you're already scrolling. Tying the visual to scroll position means the explanation moves at whatever pace you're reading, with nothing extra to operate.

A band across the middle of the viewport decides which step counts as current. The other two steps fade back rather than disappearing, so you can still see how many there are.

Q. Why do the nav indicator and the tab pill slide?

They're one element moving rather than two elements cross-fading. The movement is doing the work: it shows you where the selection went, so you don't have to find it again.

Both use a spring rather than a fixed duration, because selection should feel like it's responding to you.

Q. Why does the price animate?

A number quietly turning into a different number is easy to miss, and the number changing is the entire point of the toggle. The old one slides up and out, the new one comes up into place, over about a quarter of a second.

The column reserves the width of that plan's larger price, so the row doesn't jump sideways while the digits change. My first version reserved a fixed width for all three plans and left an odd gap next to the single-digit prices.

Q. Why are hover effects CSS when everything else uses Motion?

A hover shouldn't cause a React render. Hover and press states are plain CSS transitions. Motion only handles things that genuinely depend on state, like presence, layout changes, scroll position and switching views. There are a lot of hoverable cards on this page and it adds up.

Q. What happens if someone has reduced motion turned on?

They still get a page that responds, just without the movement. Reveals fade without travelling, buttons keep their colour change but don't lift, the sliding indicators jump instead of gliding, and the price swaps without the vertical slide.

I handled it component by component instead of with one rule that disables animation globally. A page where nothing acknowledges your clicks feels broken in its own way, which isn't much of an improvement for the person who set the preference.

THE TRADE

It meant a second pass over every animated component. There's no shortcut if you want the reduced version to be decent rather than just present.

Pricing

Not in the brief, but the page felt unfinished without it.

Q. Why three tiers, and why those numbers?

Three is about as many as anyone compares without a table. Free removes the reason to hesitate. Team at \$9 is the one the page is selling, so it's lifted, outlined and labelled. Studio at \$18 mostly makes Team look reasonably priced, and gives agencies somewhere to go, since the brief lists them as an audience.

Yearly billing shows "2 months free" rather than a percentage. It's the same discount but easier to picture.

The footnote mentions billing only for teammates who actually opened Novi that month. For a team lead worried about paying for dormant seats, that's the most reassuring line in the section, so I put it in plain sight rather than in a FAQ.

Q. Why does "Start free" open a dialog?

There's no app behind this page, so the alternative was linking to #, which makes the most important button on the page do nothing.

Instead every "Start free", in the header, hero, mobile menu, each plan, the closing section and the command palette, opens a signup dialog with a working email field and real validation. "Talk to the team" opens a contact dialog built the same way. Nothing visible on the page is inert.

Dialogs and the command palette

Four overlays, one component.

Q. Why write the dialog yourself instead of using Radix or Headless UI?

It's one component, around 150 lines, handling the focus trap, escape to close, locking body scroll, compensating for the scrollbar disappearing so the page doesn't shift sideways, and putting focus back on whatever opened it. All four overlays use it.

Dialogs are the thing most often done badly on landing pages, so I wanted to do it properly rather than reach for a package. I checked the focus trap by tabbing through repeatedly and confirming focus never escapes, and that it returns to the trigger on close.

THE TRADE

Radix has been tested against edge cases mine hasn't, like nested dialogs and dialogs inside iframes. For four overlays on a static page I think this is the right call. In a real app I'd probably just use the library.

Q. Is a command palette not overkill for a landing page?

Probably, yes. It adds nothing for someone who never presses `⌘K`, and most visitors won't.

I put it in anyway because the people this page is aimed at use Linear and Raycast daily and it's the first thing their hands do. It's keyboard-only and adds no visual weight, so the cost of being wrong about that is close to zero.

Code

How the repository is put together.

Q. Why put the design tokens in CSS instead of a Tailwind config?

Everything lives in one stylesheet as custom properties: colours for both themes, the type scale, radii, shadows and easings. There's no `tailwind.config.js` at all.

The practical benefit is that switching themes is just swapping variables on the root element, so it's instant and never flashes. And when I need to know what a colour is, there's one file to open.

THE TRADE

The class strings in the markup get long. That's the usual Tailwind complaint and I don't have a clever answer for it beyond extracting components once a pattern repeats.

Q. Why is all the copy in one file?

It keeps the section components down to layout, which makes them short enough to read in one go. It also made it easy to see which text came from the brief and which I'd written, which mattered when I was deciding how much to change.

Q. Why draw the icons instead of installing a set?

There are fifteen of them, which felt below the point where a dependency earns its place. Drawing them on one grid with one stroke weight means they match each other, and nothing ships that isn't used.

Q. How much of the brief's copy did you change?

Almost none. The headline, the supporting line and all four feature descriptions are as written. They were already clear and consistent, and rewriting them would have been changing something that wasn't broken.

I wrote the section headings, the pricing section and the closing lines, since the brief didn't include those.

Q. Is any of it server rendered at request time?

No. Every route builds to static HTML, including the Open Graph image. There's nothing here that needs a server. The interactive parts are client components at the leaves of the tree, with the boundary as low as I could push it.

Failure cases

What happens when something doesn't work.

Q. What do you get on a URL that doesn't exist?

A 404 page in the same design, with the wordmark, a working theme toggle and links to the three main sections, on the assumption that whoever hit a dead link was probably after one of them. It returns an actual 404 status.

Q. And if something throws?

There are two boundaries. An error inside the page gets replaced by a matching error screen with a "try again" button that re-renders rather than reloading, plus a way home. If there's a digest available it's shown, since that's what you'd quote in a support message.

The second one sits above the layout, for the case where the layout is what broke. It can't use the fonts, tokens or theme script, because those are all defined in the file that just failed, so it carries its own CSS inline including dark mode and a reduced-motion rule.

Q. Does it work with JavaScript off?

The content does. The reveals start at zero opacity, which would leave the page blank without scripts, so there's a `<noscript>` block in the head that forces them visible. The board and the dialogs don't work, but everything readable is there.

Testing

What I checked, and what it turned up.

Q. How did you check the responsive behaviour?

In the browser, at each size, scrolling through the whole page. Phones at 320, 360, 375, 390 and 414 px, tablets at 600, 768, 820 and 912 px, and desktop from 1024 up to 1920 px.

I was looking for three things at each width: whether the page scrolls sideways, whether anything sticks out of its container, and whether anything tappable is under 24 px. The narrow end is where the problems are. At 320 px a board column has roughly 60 px of usable width.

Q. What had to change for small screens?

The board cards drop their comment counts below the small breakpoint, avatars go down a size, and padding tightens. The tag truncates instead of pushing the avatar out of the card, which is what actually guarantees it can't overflow at any width. Before that it was spilling out by about 18 px at 320.

I kept three columns visible rather than collapsing to one scrollable column. Collapsing would have been easier, but three columns is what makes it read as a board without anyone having to interact with it.

Q. Did checking actually find anything?

Five things, none of which I could see by looking at the page:

- The heading order jumped from H1 straight to H4, because I'd marked the board's column titles up as headings when they're interface labels.
- A text size was silently disappearing. `tailwind-merge` doesn't know about custom sizes defined in CSS, so it treated `text-display-md` as a colour class and dropped it whenever it shared a call with `text-ink`. The walkthrough headings were rendering at 16 px instead of 34. Nothing warned about it; I found it by checking the computed font size.
- A hydration mismatch from `dnd-kit`. It numbers its accessibility ids from a counter that restarts on the client, so the server and the browser disagreed on every load. Fixed by giving each board an explicit id.
- Footer links were 20 px tall, under the minimum tap target.
- A duplicate `id="pricing"`, left over when the new pricing section took the anchor the closing section already had. Invalid, and it breaks the nav link.

The deployed build scores 100 on all four Lighthouse categories, with no console errors or hydration warnings.

What I left out

The gaps, stated rather than hidden.

Q. What doesn't this do?

Nothing is submitted anywhere. The signup, contact and newsletter fields validate and move through all four states, but there's no backend. Adding one would have meant picking a provider and handling a key, which felt outside the scope of a landing page assessment.

The footer links go nowhere. There are no product, company or legal pages to link to. The nav links are different: those all resolve to real sections, because a nav link that does nothing is a bug rather than a placeholder.

There's no second page. I'd rather submit one page I'm happy with than two that are half finished.